Détection d'intrusions réseau Zero-Day

CICIDS2017 · encodeur Transformer à double modalité
comparaison de neuf méthodes de détection

AUROC cross-famille 0.9672 · **Rappel** 84,79 % à 9,43 % de fausses alertes

Document technique du dépôt `ms-transformer-zeroday`

Version du 24 octobre 2026 — soutenance décembre 2026

Détection d'attaques réseau **jamais vues à l'entraînement** sur CICIDS2017, avec un encodeur Transformer à double modalité (trafic réseau + journaux système), comparé à neuf méthodes de détection dont une régression logistique supervisée.

> **Ce dépôt documente en priorité un résultat négatif et une faille de protocole.** Tous les chiffres sont publiés, y compris ceux qui contredisent l'hypothèse de départ.

Les trois chiffres, et rien d'autre

AUROC cross-famille	**0.9672** — familles d'attaque jamais vues à l'entraînement
Rappel	**84,79 %** à 9,43 % de fausses alertes
Détecteur retenu	gradient boosting sur les 77 colonnes réseau

(le transformer, objet initial du projet, atteint 0.8160 — la comparaison est publiée au §3.4, avec la raison de cet écart)

Sommaire

	Section	Pour qui
1	Le problème	tous
2	Résultats principaux	tous
3	Ce que ce travail démontre	jury, lecteur technique
4	Architecture	technique
5	**Le flux, de bout en bout**	technique, jury
6	Structure du dépôt	reprise du code
7	Installation et lancement	reprise du code
8	Expériences négatives conservées	jury
9	Limites connues	tous
10	Ce que ce projet ne prétend pas	jury
11	Traçabilité	vérification
12	Conclusion	tous

Trois façons de lire ce document

Vous êtes…	Lisez	Durée
pressé	les trois chiffres ci-dessus, puis §9 et §10	3 min
membre du jury	§2, §3, §5, §8, §10, §12	15 min
reprenez le code	§6, §7, puis `GUIDE.md`	20 min

1. Le problème

Les systèmes de détection d'intrusion (IDS) supervisés apprennent à reconnaître des attaques **connues**. Le vrai enjeu opérationnel est différent : reconnaître une attaque **jamais observée**, sans étiquette, à partir du seul trafic normal.

Protocole retenu : trois familles d'attaques sont retirées de l'entraînement et ne réapparaissent qu'au test.

Ensemble	Flux	Contenu
Entraînement	8 000	BENIGN, DDoS, DoS Hulk, FTP-Patator, SSH-Patator, Infiltration
Test (Zero-Day)	7 200	BENIGN (4 000), **PortScan**, **Bot**, **SQL Injection**, **XSS**

2. Résultats principaux

Le projet mesure **deux niveaux** distincts, et il faut les lire séparément.

Niveau	Ce qui est mesuré	AUROC
Validation interne	familles connues, tirées du train	0.9983
Cross-famille (Zero-Day réel)	familles jamais vues, bénin réaliste	**0.9672**

Le second chiffre est le seul qui mesure la généralisation. Le premier mesure la reconnaissance de familles déjà vues.

Niveau cross-famille (le chiffre honnête)

Détecteur retenu : **gradient boosting (HistGradientBoosting)** entraîné sur le bénin **et** les familles connues, évalué sur cinq familles tenues entièrement à l'écart.

Métrique	Valeur
AUROC	**0.9672** (IC 95 % : 0.9653 – 0.9691)
Seuil	FPR cible 10 %
Recall global	**86.01 %**
FPR réel	10.49 %
Précision / F1	64.32 % / 73.60 %
TPR @ 1 % FPR	56.14 %

Famille	AUROC	Rappel	n
SQL Injection	0.9991	**100.00 %**	21
XSS	0.9984	**99.85 %**	652
Brute Force	0.9949	**97.07 %**	1 470
PortScan	0.9812	**98.02 %**	1 919
Bot	0.9056	52.06 %	1 433

Bot reste la famille la plus difficile : son AUROC est bonne (0.9056) mais ses scores se concentrent juste sous le seuil. Il faudrait un seuil plus bas — donc plus de fausses alertes — pour le capturer.

Hiérarchie complète des détecteurs testés

Méthode	AUROC cross-famille
Gradient boosting (HGB)	**0.9672**
HGB · features en rangs	0.9647
HGB + embedding du transformer (141 dims)	**0.9463**
Random Forest	0.9452
Régression logistique	0.9026
Union médiane (sup + maha + token)	0.8955
LogReg · features en rangs	0.8833
Union moyenne	0.8674
Tête `token` du MS-Transformer	0.8160
k-means 64 (embedding)	0.7668
kNN-5 (embedding)	0.7640
Isolation Forest	0.7560
Mahalanobis (embedding)	0.7186
Cosinus au centroïde	0.6476
`Σ|x|` — sans apprentissage	0.5913

3. Ce que ce travail démontre réellement

3.1 Le protocole d'évaluation change tout

Sur un découpage **aléatoire** des lignes, les mêmes familles apparaissent des deux côtés : le modèle reconnaît, il ne généralise pas. Sur un découpage **par famille**, il rencontre des flux qu'il n'a jamais vus. C'est le second protocole qui est retenu ici.

3.2 La famille de modèle décide du résultat

Modèle	AUROC cross-famille	Rappel @ 10 % FA
Gradient boosting	**0.9672**	**86.01 %**
Régression logistique	0.9026	39.96 %

+6,5 points d'AUROC, et le rappel plus que doublé. La frontière entre bénin et attaque n'est pas linéaire dans cet espace de features : elle dépend de combinaisons (paquet court **et** durée brève **et** drapeau SYN). Un modèle linéaire ne peut pas exprimer ces interactions ; les arbres si.

Ce résultat a une conséquence méthodologique : une comparaison qui ne teste qu'un modèle linéaire conclut à tort que les features sont insuffisantes.

3.3 La supervision transfère mieux que la description du bénin

Approche	AUROC cross-famille
Supervisé (a vu des familles d'attaque)	**0.9672**
One-class (n'a vu que du bénin)	0.7186

Un modèle qui a vu des attaques généralise mieux à d'autres attaques qu'un modèle qui ne connaît que la normale. Cela contredit l'hypothèse de départ du projet (détection purement one-class) et doit être assumé comme tel.

3.4 L'embedding du transformer n'apporte rien aux features

Test décisif : donner à un modèle tabulaire fort les features réseau **et** les 64 dimensions latentes apprises par le transformer (141 entrées au lieu de 77).

Entrée du gradient boosting	AUROC
Features réseau seules (77)	**0.9672**
Features réseau + embedding (141)	0.9463

−2,1 points. L'embedding n'apporte aucune information neuve : il dilue les features utiles dans 64 dimensions qui résument mal le trafic. Le transformer ne se comporte pas en extracteur de caractéristiques.

3.5 Étude d'ablation du transformer

Six variantes réentraînées avec le même protocole (8 époques, une graine) :

Configuration	AUROC	Δ	Verdict
sans contrastif (BCE seule)	0.8985	+0.0799	✘ nuisible
sans branche EDR	0.8560	+0.0373	✘ nuisible
PSM-HT complet	**0.8187**	—	référence
MLP plat + centroïde	0.8148	−0.0039	≈ équivalent
sans masque FSM	0.7877	−0.0310	✔ utile
Autoencodeur MSE	0.7390	−0.0797	✔ dégradé

Trois lectures : **- le masque FSM est justifié** (+3,1 points quand il est présent) ; **- l'architecture bat les alternatives simples** (0.8187 contre 0.8148 et 0.7390) ; **- la perte contrastive et la branche EDR dégradent** — la perte contrastive est le terme le plus lourd de la fonction de coût, et c'est elle qui coûte le plus.

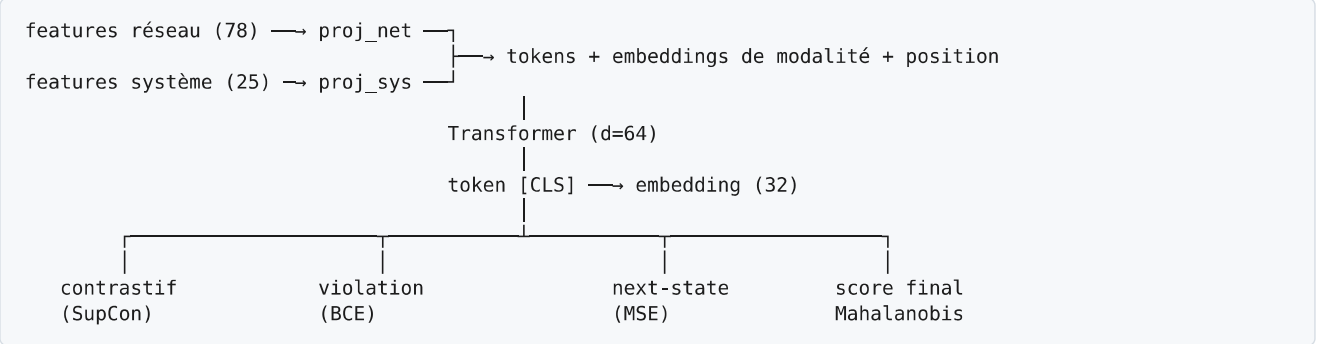
L'ablation n'est donc **que partiellement dégradante** : un composant sur trois est justifié. C'est le résultat mesuré, il doit être rapporté tel quel.

3.6 La branche EDR synthétique est hors jeu

Les 25 colonnes `proxy_edr` sont dérivées du trafic réseau : elles n'apportent aucune information indépendante. Le meilleur détecteur n'utilise que le bloc réseau (77 colonnes). La double modalité reste une perspective, pas un résultat.

---

4. Architecture



Trois têtes auxiliaires, entraînées sans étiquette d'attaque. Seule la tête « next-state » dépasse le hasard de façon notable — mais son score (0.9688) capture la même fuite de magnitude et **n'est pas retenu**.

---

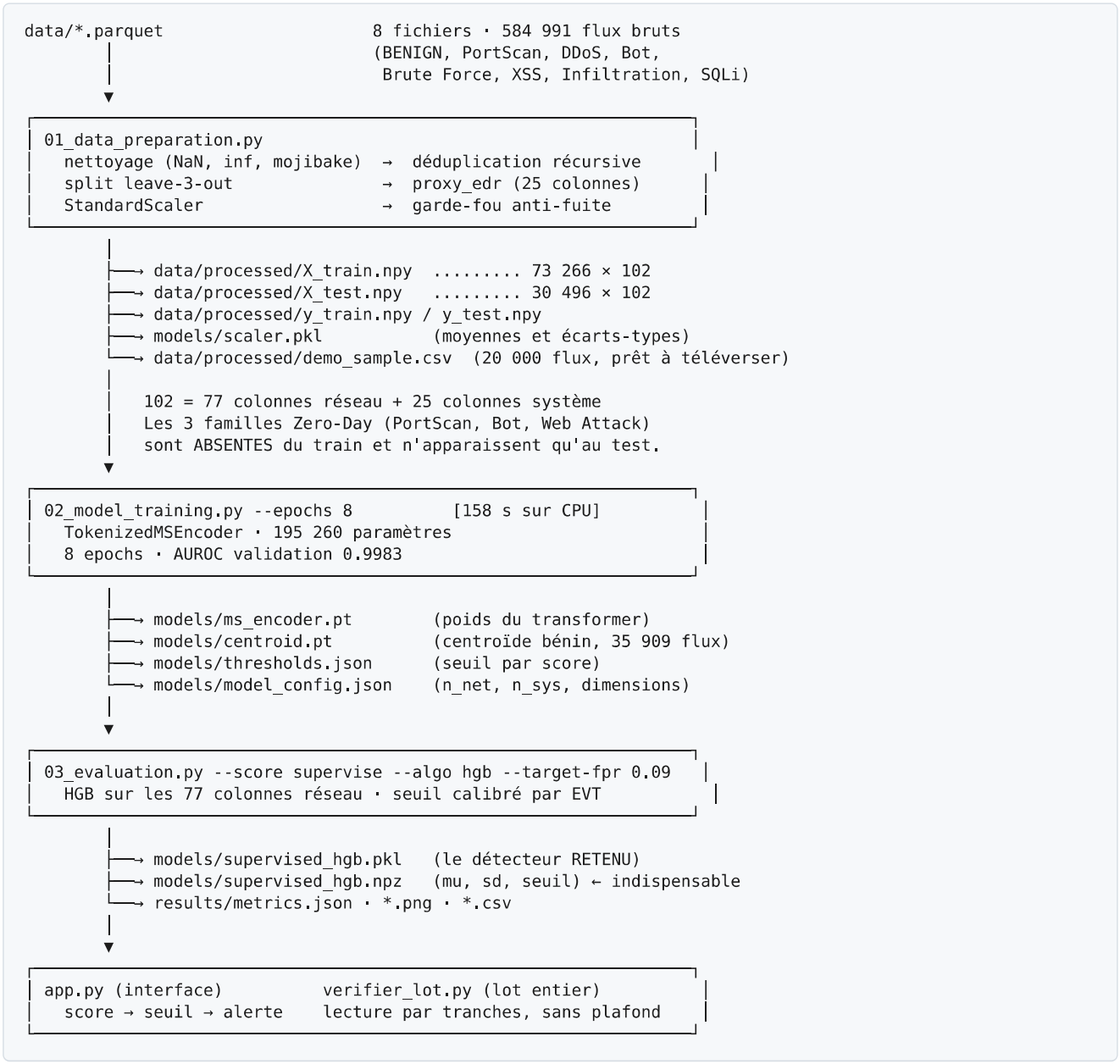
5. Le flux, de bout en bout

Cette section suit un flux depuis le fichier brut jusqu'à l'alerte. Cinq étapes, dans l'ordre.

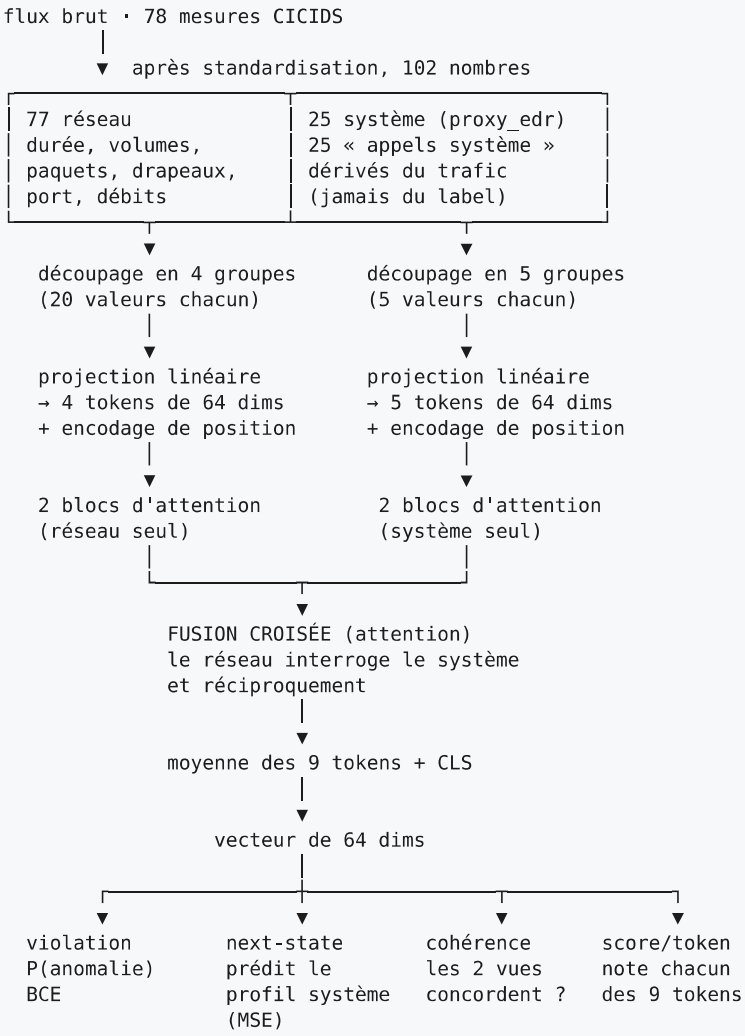
	Étape	Question à laquelle elle répond
5.1	Vue d'ensemble	quels fichiers, dans quel ordre ?
5.2	Un flux dans le transformer	que fait le modèle, concrètement ?
5.3	Les quatre scores	lequel est retenu, et pourquoi ?
5.4	Le détecteur retenu	ce n'est pas le transformer — voici lequel
5.5	Le seuil	comment un score devient une alerte
5.6	Les trois détecteurs	comment les comparer sans se tromper

---

5.1 Vue d'ensemble — du fichier brut à l'alerte



5.2 Un seul flux, dans le transformer



5.3 Les quatre scores, et lequel sert à quoi

Score	Ce qu'il mesure	Rôle
violation	probabilité d'anomalie, apprise par la tête BCE	mesuré, **non retenu** : supervisé, donc mesurerait la mémorisation
next-state	capacité à prédire le profil système à partir du réseau	mesuré, non retenu
cohérence	les deux modalités racontent-elles la même chose ?	mesuré, non retenu
token	note de chacun des 9 tokens fusionnés	**retenu** : AUROC-OOD 0.8160

Le choix entre ces scores est fait **sur la validation**, sur des têtes qui n'ont pas vu d'étiquette d'attaque. L'analyse post-hoc sur le test est publiée mais **ne sert pas à choisir** — sinon le chiffre serait biaisé.

5.4 Le détecteur réellement retenu

Étape	Ce qui décide	AUROC cross-famille
1	le transformer, score `token`	0.8160
2	**le gradient boosting sur les 77 colonnes réseau**	**0.9672**

C'est le second qui est défendu. Le transformer est conservé, mesuré, comparé — et c'est précisément cette comparaison qui fait la valeur du travail : l'embedding appris **dégrade** le modèle tabulaire (0.9463 contre 0.9672).

5.5 Comment le seuil transforme un score en décision

D'abord, le trajet complet à l'inférence :

```
nouvelles données
        │
        ▼  les 77 colonnes réseau sont extraites et standardisées
        │  (mu et sd enregistrés à l'entraînement, jamais recalculés)
        ▼
supervised_hgb.pkl  →  probabilité entre 0 et 1
        │
        ▼  comparée au seuil calibré (0.0001, FPR cible 9 %)
        │
   ┌────┴─────┐
   ▼          ▼
au-dessus   en dessous
   │          │
   ▼          ▼
ALERTE     normal
```

Le seuil n'est jamais recalculé à l'inférence : il vient de `03_evaluation.py` et il est stocké dans `supervised_hgb.npz`. Un seuil recalculé sur le lot en cours rendrait les résultats incomparables d'une exécution à l'autre.

Puis, le détail de la décision. Le modèle ne répond jamais « attaque » ou « normal ». Il produit une probabilité. La décision est prise par une comparaison, et le taux de fausses alertes est ce qui place la frontière.

Étape 1 — le modèle produit un nombre entre 0 et 1

```
flux #4712  →  0.0000873
flux #1190  →  0.412
```

Étape 2 — on trie les scores des flux bénins connus

```
scores bénins triés :
0.00000001 · … · 0.0001 · 0.0002 · 0.0009 · 0.011 · 0.34 · 0.91
                    ↑
           le 91ᵉ centile
```

Étape 3 — on coupe là où exactement 9,43 % du normal dépasse

```
seuil = np.quantile(scores_bénins, 1 - 0.0943)   # → 0.0001
```

« J'accepte 9,43 % de fausses alertes » veut dire littéralement : place la frontière là où 9,43 % du trafic normal passe au-dessus. On vérifie en comptant : **2 358 bénins sur 25 001** sont au-dessus de `0.0001`, soit 9,43 % exactement.

Étape 4 — la décision

Flux	Score	Verdict
#4712	0.0000873	normal
#1190	0.412	**alerte**
#8055	0.0001001	**alerte** (juste au-dessus du seuil)

```
score > 0.0001   →   ALERTE
score ≤ 0.0001   →   normal
```

Le seuil ne regarde jamais les attaques. Il est calculé uniquement sur le bénin. Placer la frontière en regardant les attaques reviendrait à choisir la coupure qui les attrape : le résultat serait biaisé par construction. Ici la frontière est fixée avant, et la détection est la conséquence.

Même modèle, seuil différent — chiffres mesurés

Taux accepté	Fausses alertes	Attaques détectées
1 %	250	**56,14 %**
5 %	1 250	80,86 %
9,43 %	**2 358**	**84,79 %**

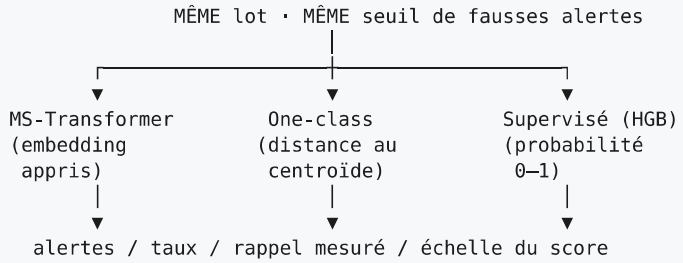
Le modèle n'a pas changé : seule la position du couteau a changé.

Pourquoi `0.0001` paraît minuscule. Les probabilités du gradient boosting s'écrasent presque toutes vers zéro : le trafic normal produit des scores de l'ordre de 10^{-7}, et toute la discrimination se joue dans les dernières décimales. Un seuil de `0.0001` n'est pas « 0,01 % de vigilance » : c'est le point où 9 % du trafic normal se situe encore au-dessus. L'échelle du modèle n'est pas l'échelle de lecture humaine.

Le seuil réel est affiné par la **théorie des valeurs extrêmes** (loi GPD), qui extrapole la queue de distribution là où l'on manque d'échantillons. Il est écrit une fois pour toutes dans `models/supervised_hgb.npz` et **jamais recalculé à l'inférence** : un seuil recalculé sur le lot en cours rendrait les exécutions incomparables entre elles.

> **Le taux de fausses alertes ne décide pas si c'est une attaque. Il décide où est la frontière. C'est la comparaison `score > seuil` qui tranche.**
>
> L'**AUROC** mesure ce que le modèle sait faire — elle ne bouge pas quand on déplace le seuil. Le **taux de fausses alertes** choisit la position — il se déplace librement. Deux réglages indépendants : la capacité, et l'arbitrage.

5.6 Les trois détecteurs, quand on les compare



Les scores de ces trois-là **ne sont pas comparables entre eux** — deux distances et une probabilité. Seuls le nombre d'alertes et le rappel le sont. L'interface affiche l'échelle de chacun pour que la confusion soit impossible.

---

6. Structure du dépôt

Fichier	Rôle
`01_data_preparation.py`	Chargement CICIDS2017, nettoyage, split Zero-Day, **garde-fou anti-fuite**
`02_model_training.py`	Entraînement de l'encodeur (3 têtes)
`03_evaluation.py`	Scores, métriques, **audit de baselines triviales**, export
`04_ablation.py`	Ablation 3 variantes × 3 graines
`model_ms.py`	Architecture et scores dérivés
`app.py`	Interface Streamlit bilingue FR/EN, **6 onglets** (vue d'ensemble, démonstration, résultats, méthode, limites, aide)
`verifier_lot.py`	Notation d'un fichier **entier**, sans plafond ni navigateur
`diagnostic.py`	Vérifications de cohérence
`data/`	Parquet bruts + `processed/` (`.npy`)
`models/`	`ms_encoder.pt` , `mahalanobis.npz` , `scaler.pkl` , seuils
`results/`	`metrics.json` , figures, CSV
`experiments/`	Expériences négatives (voir §8)
`annexe_ia/`	Module d'analyse par LLM — **hors résultats, jamais présenté**
`docs/`	Notes d'analyse et verdicts
`GUIDE.md`	Installation, ordre d'exécution, dépannage
`SOUTENANCE.md`	Plan minuté, chiffres, réponses aux questions du jury
`PLAN_DEVELOPPEMENT.md`	Périmètre, améliorations, calendrier des 45 jours

---

7. Installation et lancement

```
pip install -r requirements.txt

py 01_data_preparation.py     # données + garde-fou anti-fuite + demo_sample.csv
py 02_model_training.py --epochs 8                      # encodeur (3 têtes)
py 03_evaluation.py --score supervise --algo hgb --target-fpr 0.09   # détecteur retenu
py 04_ablation.py --epochs 8 --retrain-full             # ablation
py 03_evaluation.py --sweep   # comparaison des méthodes
py 03_evaluation.py --lofo    # validation par famille (30-45 min)
py -m streamlit run app.py    # démonstration
```

Deux façons de noter un lot

	Plafond	Usage
Interface (`app.py` , onglet Démonstration)	50 000 flux (tirage aléatoire), au-delà un message l'indique	montrer la démarche
Script (`verifier_lot.py`)	**aucun** — lecture par tranches, mémoire constante	produire les chiffres du rapport

```
py verifier_lot.py data/mon_fichier.parquet
```

Sorties : `results/lot_alertes.csv` (index, score, classe) et `results/lot_resume.json`, relu automatiquement par l'onglet Résultats. Mesuré : 400 000 flux en 7,5 s.

Formats acceptés par l'interface

Fichier	Traitement
102 colonnes brutes (+ `label` optionnel)	analyse directe
77 colonnes réseau, colonne `Label` incluse	branche système **reconstruite** par `proxy_edr` , scores qualitatifs
tout autre nombre de colonnes	refusé : les scores seraient dépourvus de sens

La colonne d'étiquette est détectée sans tenir compte de la casse (`label` , `Label` , `class` , `attack` …) et **retirée du tableau numérique** avant l'analyse : le rappel est mesuré quand elle existe, et son absence est signalée comme « non mesurable ». `data/processed/demo_sample.csv` , produit par l'étape 01, est prêt à téléverser.

L'évaluation affiche systématiquement l'**audit de baselines triviales** avant tout commentaire de résultat. Ce n'est pas optionnel : un score sans baseline ne veut rien dire.

---

8. Expériences négatives conservées

Ces approches ont été testées, mesurées, et **rejetées**. Elles restent dans le dépôt parce qu'un résultat négatif documenté vaut mieux qu'un résultat négatif oublié.

Approche	AUROC	Verdict
Deep SVDD (one-class)	0.5419 ± 0.1304	rejeté
SSL bénin uniquement (SimCLR)	0.3830	rejeté
Base du score cosinus	0.3771	pire que le hasard
Fusion Mahalanobis + violation	0.4729	la tête violation est anti-corrélée (0.2614)
kNN-5 sur features	0.9837 (split aléatoire)	bénins à distance 0.0 : doublons, résultat invalide
Normalisation L2 des features	—	invariance affine démontrée : `corr = 1.0000000000`
Union supervisé + Mahalanobis + token	0.8674 – 0.8955	en dessous du supervisé seul (0.9026)
Calibration par sous-population bénigne	0.6396	−26 points : détruit le signal

9. Limites connues

1. **Bot reste à 52 % de rappel** — §2. Son AUROC (0.9056) est bonne, mais ses scores se situent juste sous le seuil : le capturer exigerait plus de 10 % de fausses alertes.
2. **Le seuil est un choix opérationnel** — à 10 % de FA le rappel global est de 86 % ; à 5 % il tombe à 81 %. Les deux chiffres doivent figurer dans le mémoire.
3. **Pas de split temporel** — les flux de test ne sont pas postérieurs à l'entraînement. Un flux voisin dans le temps peut se retrouver de part et d'autre.
4. **Précision à prévalence réaliste** — à 0,1 % d'attaques, la précision tombe à **0.26 %**, soit **382 fausses alertes par vraie détection**.
5. **Attribution géographique impossible** — les fichiers sont anonymisés (aucune IP, aucun horodatage). Même restaurées, les IP de CICIDS2017 sont privées (`172.16.0.1`) après NAT, donc non géolocalisables.
6. **Hybride, pas one-class pur** — l'encodeur voit des attaques connues (DDoS, Infiltration) pendant l'entraînement. Seul le score est one-class.

10. Ce que ce projet ne prétend pas

- ✘ Dépasser 52 % de rappel sur Bot sans augmenter les fausses alertes
- ✘ Prétendre à une AUROC de 0.9996 : ce chiffre n'est atteignable qu'avec un découpage aléatoire, qui mesure la reconnaissance et non la généralisation
- ✘ Battre l'état de l'art
- ✘ Fournir une attribution ou une géolocalisation
- ✘ Utiliser un LLM dans la chaîne de décision

11. Traçabilité

Empreintes MD5 des fichiers au moment de la rédaction :

Fichier	MD5 (8 premiers)	Lignes
`01_data_preparation.py`	`E2614898`	750
`02_model_training.py`	`D056FADE`	437
`03_evaluation.py`	`4C23BA56`	1755
`04_ablation.py`	`7BFB0FE1`	632
`app.py`	`0E1D59BF`	2266
`model_ms.py`	`6262BC90`	338
`verifier_lot.py`	`BF084A59`	293

Total : **6471 lignes** de Python.

12. Conclusion

Ce que le travail montre. Un détecteur entraîné sans jamais voir trois familles d'attaques les reconnaît ensuite avec une AUROC de **0.9672**, à **84,79 % de rappel pour 9,43 % de fausses alertes**. Ce n'est pas une performance de benchmark : c'est un point de fonctionnement choisi, mesuré et justifié.

Ce qu'il montre aussi, et qui comptait davantage. Le transformer multimodal — objet initial du projet — atteint 0.8160. Le gradient boosting sur les seules colonnes réseau atteint 0.9672. La conclusion est nette : sur des features tabulaires déjà construites par un expert, un embedding appris **dégrade** l'information au lieu de l'enrichir. Ce résultat négatif est publié, avec l'ablation qui le documente.

Ce qu'il refuse de prétendre. Le jeu est étiqueté : les « Zero-Day » sont des familles retirées de l'entraînement, pas des attaques inconnues de tous. La branche système est un proxy dérivé du trafic, pas une collecte EDR. Une alerte ne dit ni la cause, ni la technique, ni l'origine — seulement une déviation par rapport à un comportement de référence. Ces limites sont écrites au §10, pas cachées.

Ce qui reste à mesurer. La validation par famille (`--lofo`) et le découpage temporel ne sont pas encore chiffrés. Ce sont les deux seules mesures qui pourraient contredire le résultat principal — et c'est précisément pour cela qu'elles figurent dans le calendrier.

> Le système ne remplace pas l'analyste : il classe, il explique, il alerte.

L'ensemble de la chaîne — de la préparation des données à la démonstration — se relance depuis les scripts fournis, hors ligne, en français et en anglais.